

Placement de vidéos dans des serveurs cache

Hash Code 2017 — Streaming Videos

Solveur glouton à gain marginal, knapsack 0/1 et fast-fill

Objectif : minimiser la latence moyenne des requêtes
en plaçant des copies de vidéos dans des caches de capacité limitée.

Fichiers : main.py · score.py · generate_instances.py
Instances officielles + 8 instances générées (graine 20260913)

Document de synthèse du fonctionnement de l'algorithme
et des expériences sur instances officielles et générées.

Sommaire

1. Introduction et objectif
2. Énoncé du problème Hash Code
3. Fonction de score
4. Idée centrale : le gain marginal
5. Architecture du solveur
6. Chemin petit / moyen (greedy, knapsack, reopt, hill-climb)
7. Chemin grand (fast-fill)
8. Résultats sur les instances officielles
9. Sur quels types d'instances l'algorithme est efficace
10. Instances générées et résultats expérimentaux
11. Conclusion

Ce compte rendu décrit le fonctionnement du solveur implanté dans `main.py`, le calcul du score officiel, puis les expériences menées sur les quatre fichiers Hash Code et sur huit instances synthétiques générées par `generate_instances.py`.

1. Introduction et objectif

Le problème traité est celui de Google Hash Code 2017, qualification, Streaming Videos. Un réseau de vidéo à la demande dessert des utilisateurs depuis un datacenter unique, trop loin. On dispose de serveurs cache, plus proches, mais de capacité limitée. Il faut décider quelles vidéos copier dans quels caches pour réduire le plus possible le temps d'attente moyen.

Le problème est un placement combinatoire : chaque cache est un sac à dos, mais les sacs interagissent. Une requête n'utilise que le cache le plus rapide qui contient déjà la vidéo. Dupliquer une vidéo dans un cache plus lent n'apporte donc rien. C'est cette contrainte, plus que la capacité seule, qui rend un greedy naïf inefficace.

L'objectif du projet n'est pas un algorithme exact (NP-difficile, instances jusqu'à 10 000 vidéos et 200 000 requêtes), mais un heuristique rapide et robuste : bon sur les petites instances, et capable de terminer sur les très grandes dans le temps d'un juge en ligne (Sphere Engine).

2. Énoncé du problème Hash Code

Une instance est un graphe biparti entre endpoints et caches, plus un datacenter relié à tout le monde.

Modèle du réseau de streaming

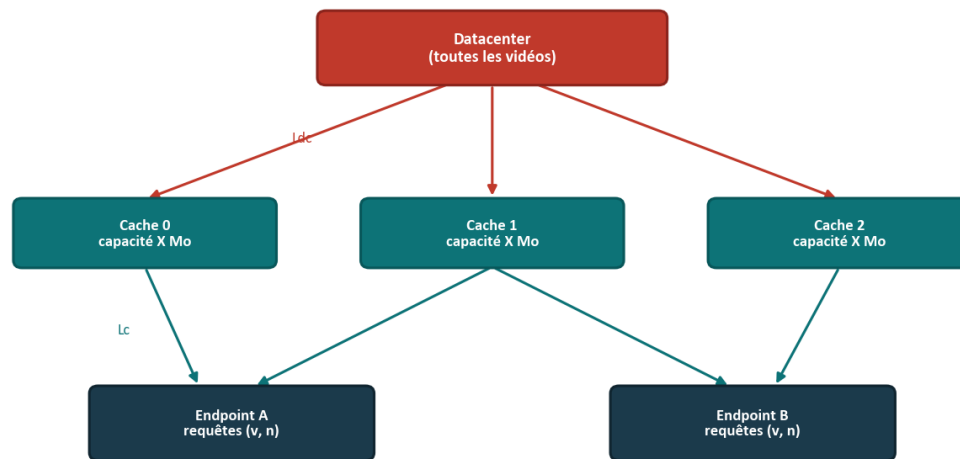


Figure 1 — Datacenter, caches de capacité X, endpoints et requêtes.

Données d'entrée

La première ligne d'un fichier `.in` donne `V E R C X` : nombre de vidéos, d'endpoints, de descriptions de requêtes, de caches, et capacité d'un cache en Mo. Viennent ensuite les tailles des vidéos, puis pour chaque endpoint sa latence vers le datacenter et ses liens vers les caches (identifiant, latence), enfin les requêtes (vidéo, endpoint, nombre).

Symbole	Signification
V, taille $s(v)$	Catalogue : chaque vidéo a une taille en Mo
C, capacité X	Tous les caches ont la même capacité X
E, $Ldc(e)$	Endpoints et latence vers le datacenter
$Lc(e, c)$	Latence de l'endpoint e vers le cache c (s'il est relié)
R, $n(v, e)$	Volume de requêtes de la vidéo v depuis e

Décision et contraintes

Pour chaque cache c , on choisit un sous-ensemble de vidéos $S(c)$ tel que la somme des tailles ne dépasse pas X. Une vidéo peut être copiée dans plusieurs caches, ou dans aucun. Le datacenter contient toujours tout le catalogue : une requête est toujours servie, au pire avec Ldc .

Pour une requête (v, e) , la latence réelle est le minimum entre $Ldc(e)$ et toutes les $Lc(e, c)$ des caches c qui contiennent v et qui sont reliés à e . Seul le meilleur cache compte.

3. Fonction de score

Le score officiel, implanté dans `score.py`, est le temps économisé moyen, en microsecondes par requête, arrondi à l'entier inférieur :

$$\text{score} = \left(\sum (Ldc(e) - L^*(v, e)) \times n(v, e) \times 1000 \right) / \sum n(v, e)$$

où $L^*(v, e)$ est la meilleure latence après placement. Si aucun cache utile n'a la vidéo, $L^* = Ldc$ et le gain est nul. Un réseau sans aucun lien cache donne donc forcément 0 — c'est le test de santé `no_cache`.

Cette formule pèse les requêtes par leur volume. Une vidéo très demandée sur un endpoint lent rapport beaucoup plus qu'une vidéo rare. C'est pourquoi l'algorithme trie les placements par densité = gain / taille.

4. Idée centrale : le gain marginal

Un greedy naïf compare chaque couple (cache, vidéo) au datacenter, indépendamment des autres caches. Il place alors la même vidéo populaire partout, même dans des caches plus lents. Le score officiel n'en profite pas : seule la copie la plus proche est utilisée. La capacité, elle, est gaspillée.

Le solveur calcule donc un gain marginal. Pour chaque couple (vidéo, endpoint), on mémorise la meilleure latence déjà atteinte `current_best`. Placer la vidéo dans un nouveau cache ne rapporte que la différence avec ce meilleur actuel, et seulement si le nouveau cache est plus rapide.

Gain marginal : seul le cache le plus rapide compte

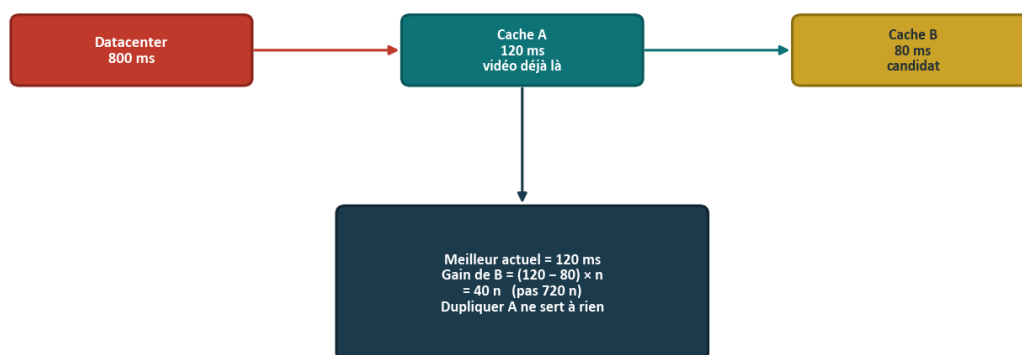


Figure 2 — Une fois la vidéo dans A (120 ms), B ne vaut plus que $40 \text{ ms} \times n$, pas $720 \text{ ms} \times n$.

Conséquence pratique : dès qu'une bonne copie existe, les candidats plus lents tombent à gain 0 et sont ignorés. On évite la duplication inutile. C'est particulièrement décisif sur `trending_today`, où tous les caches ont la même latence : une seule copie par vidéo suffit pour tout le réseau.

5. Architecture du solveur

Tout passe par `solve`. Selon la taille, on emprunte l'un des deux chemins.

Choix du chemin algorithmique

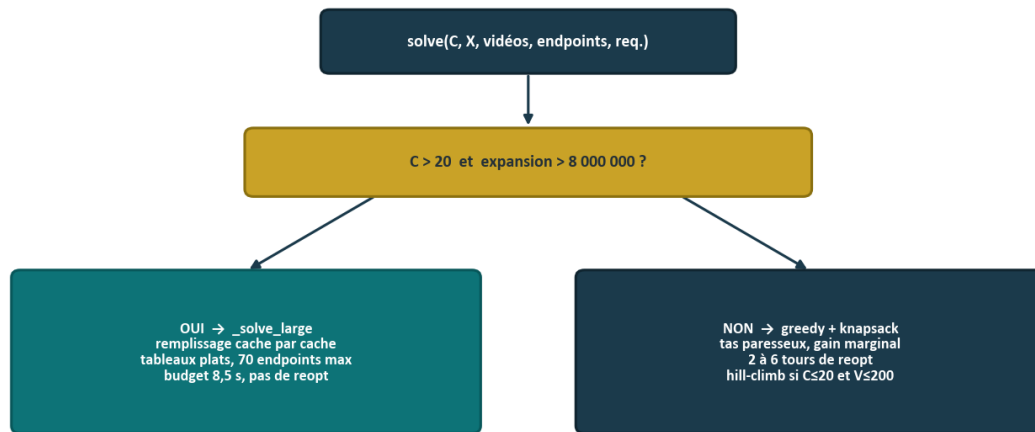


Figure 3 — Le test d'expansion évite de construire un tas de plusieurs millions de candidats sur kittens.

L'expansion est une estimation du travail : on additionne, pour chaque requête, le nombre de caches visibles par son endpoint. Si $C > 20$ et que cette somme dépasse 8 millions, on bascule vers `_solve_large`. Sinon on reste sur le greedy global, plus précis.

Les entrées-sorties sont adaptées au juge : sans argument, `main.py` lit stdin et écrit uniquement la solution sur stdout (aucun log). En local : `python main.py fichier.in fichier.out`. Les messages de debug vont sur stderr.

6. Chemin petit / moyen

Pipeline du solveur petit / moyen



Figure 4 — Cinq étapes du solveur précis (zoo, videos, trending, petites générées).

6.1 Agrégation des requêtes

`_aggregate_requests` fusionne les lignes qui portent sur le même couple (vidéo, endpoint). On obtient un dictionnaire de volumes et, pour chaque vidéo, la liste des endpoints qui la demandent. Toutes les étapes suivantes raisonnent sur ces agrégats.

6.2 Glouton à tas paresseux

On initialise un tas max de candidats (cache, vidéo) scorés contre le datacenter, triés par densité $-\text{gain}/\text{taille}$. Tant que le tas n'est pas vide :

1. On extrait le meilleur candidat.
2. On recalcule son gain marginal avec `current_best`.
3. Si le gain a changé (une autre copie a déjà amélioré ces endpoints), on réinsère le candidat avec la nouvelle valeur — d'où le nom de tas paresseux.
4. Si le gain est toujours le même et positif, on place la vidéo et on met à jour `current_best`.

Cette stratégie est un greedy incrémental classique : à chaque pas on prend le placement de meilleure densité au vu de l'état courant, sans tout recalculer à chaque fois.

6.3 Knapsack 0/1 par cache

Après le greedy, chaque cache est réoptimisé comme un sac à dos 0/1, les autres caches étant figés. La valeur d'une vidéo est son gain marginal actuel, le poids est sa taille. `_knapsack_select` utilise une DP classique en $O(n \times X)$ si la mémoire reste raisonnable ($X \leq 20\,000$ et $n \times X \leq 1\,500\,000$). Sinon on retombe sur un greedy par densité, pour éviter un dépassement mémoire sur le juge (SIGSEGV observé avec une DP trop grande).

On vide le cache, on recalcule les meilleurs latences sans lui, on résout le knapsack, on remplace. C'est un tour de reoptimisation coordonnée. On fait 6 tours si l'instance est petite ($C \leq 20$ et $V \leq 200$), 2 tours sinon. On s'arrête dès qu'un tour ne change plus rien.

6.4 Hill-climb (petites instances seulement)

Sur zoo et les toutes petites instances, on lance encore une recherche locale `_hill_climb_small` : essayer d'ajouter une vidéo s'il reste de la place, remplacer une vidéo par une autre, ou déplacer une vidéo d'un cache vers un autre. On accepte uniquement si le score officiel augmente. Au plus 20 passes. Trop coûteux dès que C ou V grandit, donc désactivé ailleurs.

7. Chemin grand : fast-fill

Sur kittens (10 000 vidéos, 1 000 endpoints, 200 000 requêtes, 500 caches), le tas global exploserait. `_solve_large` remplit les caches un par un, du plus demandé au moins demandé.

Les requêtes sont stockées en tableaux plats indexés par un identifiant `rid` : vidéo, endpoint, volume, meilleure latence courante. Chaque endpoint et chaque vidéo connaît la liste de ses `rid`. Pour un cache, on ne parcourt que ses endpoints (au plus 70, les plus volumineux) et on accumule le gain de chaque vidéo. On trie par densité et on glisse les vidéos tant qu'il reste de la place, puis on met à jour `req_best`.

Un budget de 8,5 secondes coupe la boucle si le juge est trop strict. Il n'y a pas de knapsack ni de reopt : on privilégie une solution valide et scorée plutôt qu'un optimum trop lent (un run complet sans limite atteint environ 967 000 sur kittens, mais en plusieurs minutes — 0 point sur Sphere Engine).

8. Résultats sur les instances officielles

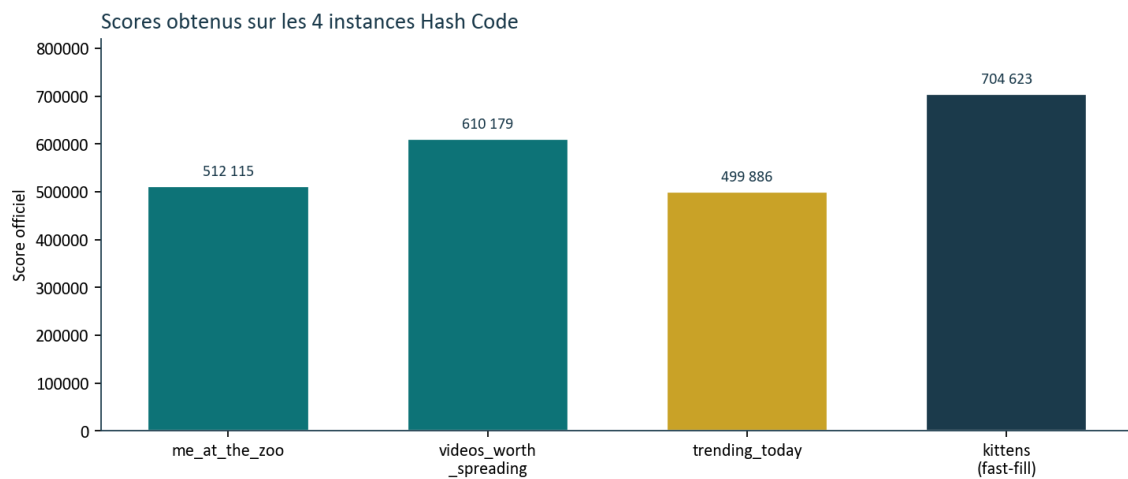


Figure 5 — Scores locaux du solveur actuel (kittens en mode fast-fill).

Instance	V	E	R	C	X	Chemin	Score
me_at_the_zoo	100	10	100	10	100	greedy + DP + hill-climb	512 115
videos_worth_spreading	10 000	100	100 000	100	10 000	greedy + reopt	610 179
trending_today	10 000	100	100 000	100	50 000	greedy + reopt	499 886
kittens (rapide)	10 000	1 000	200 000	500	6 000	fast-fill 8,5 s	704 623
kittens (lent, hors juge)	10 000	1 000	200 000	500	6 000	solveur complet	≈ 967 540

Le total local avec kittens rapide est d'environ 2,33 millions. Sur Sphere Engine, une soumission antérieure (avant le fast-fill) a obtenu 1 622 260 points et la 47^e place : zoo + videos + trending, kittens à 0 faute de temps. Le fast-fill vise précisément à récupérer plusieurs centaines de milliers de points sur cette dernière instance.

Sur trending, le score est essentiellement maximal : les latences cache sont identiques, donc une copie bien choisie suffit. Sur zoo, le hill-climb rapproche le score du plafond. Sur videos_worth_spreading, beaucoup de caches se recouvrent et la capacité est trop petite pour le catalogue : une partie du manque est structurelle, pas seulement algorithmique.

9. Sur quels types d'instances l'algorithme est efficace

Pour comparer des instances de tailles différentes, on mesure le rapport entre le score obtenu et un plafond structurel : chaque requête utiliserait le cache relié le plus rapide, sans tenir compte de la capacité. Ce n'est pas l'optimum vrai (souvent impossible), mais cela indique si l'algo capte le gain « facile » du réseau.

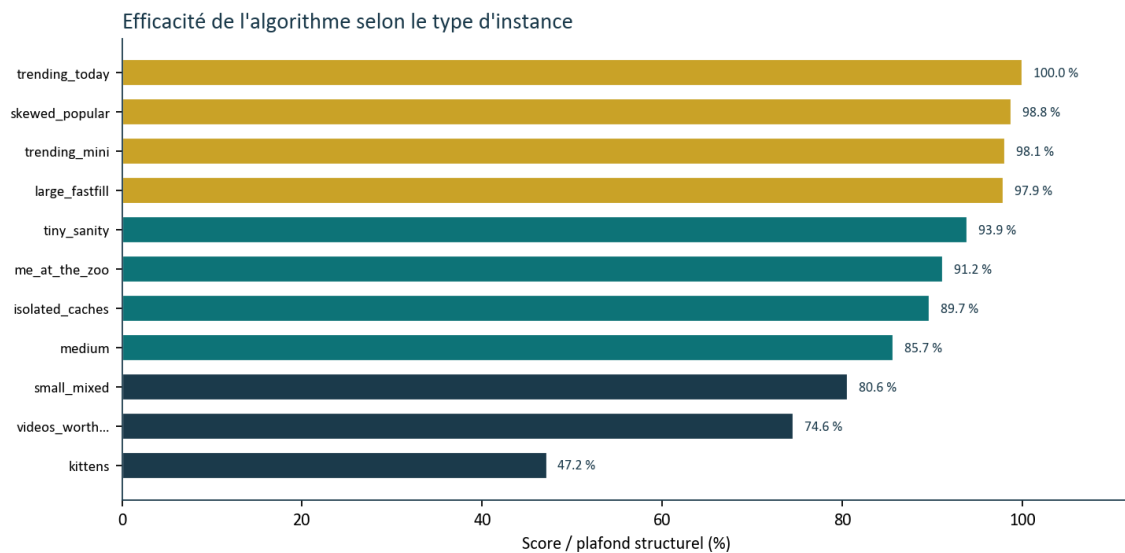


Figure 6 — Du plus adapté (trending, popularité concentrée) au plus difficile (kittens).

L'algorithme est très à l'aise quand les caches ont la même latence (trending), quand quelques petites vidéos portent presque tout le trafic (loi de Zipf, skewed_popular), et quand l'instance est assez petite pour le knapsack exact et le hill-climb (zoo). Il reste bon si les caches sont isolés : chaque sac à dos est presque indépendant.

Il est moins à l'aise dès que le graphe est immense et dense (kittens) : le plafond à 70 endpoints, le budget temps et l'absence de reopt laissent environ la moitié du plafond sur la table. Les instances à fort recouvrement et capacité serrée (videos_worth_spreading) sont intermédiaires : le greedy incrémental aide, mais le vrai optimum resterait très coûteux.

10. Instances générées

Les quatre fichiers officiels ne suffisent pas pour comprendre pourquoi l'algorithme réussit ou échoue. Le script `generate_instances.py` fabrique huit fichiers `.in` au format Hash Code, avec une graine fixe 20260913 pour que les expériences soient reproductibles. Commandes :

```
python generate_instances.py          # écrit generated/*.in
python generate_instances.py --run    # résout et affiche les scores
```

Chaque preset a un rôle : santé, cas extrême, ou caricature d'une instance officielle. Les requêtes suivent souvent une loi de Zipf (quelques vidéos très demandées), ce qui est réaliste pour de la VOD.

Fichier	V	E	R	C	X	Score	Effic.
tiny_sanity	6	3	8	2	40	601 910	93,9 %
no_cache	10	4	20	3	100	0	—
isolated_caches	40	8	120	8	80	597 826	89,7 %
trending_mini	80	8	400	6	200	549 166	98,1 %
small_mixed	60	12	180	8	120	435 295	80,6 %
skewed_popular	120	20	600	12	250	641 298	98,8 %
medium	400	35	4 000	20	800	510 403	85,7 %
large_fastfill	800	60	90 000	45	1 500	621 021	97,9 %

Tableau 1 — Dimensions et scores des instances générées (graine 20260913).

10.1 tiny_sanity — vérification à la main

Six vidéos, trois endpoints, deux caches de 40 Mo. Les tailles et les requêtes sont écrites en dur, sans aléa. On peut recalculer le score sur papier et confirmer que `lecture`, `placement` et `score.py` sont cohérents. Score : 601 910 (93,9 % du plafond).

10.2 no_cache — test de santé

Les endpoints n'ont aucun lien vers un cache. Quelle que soit la solution, toutes les requêtes restent au datacenter. Le score doit être exactement 0. Si ce n'est pas le cas, le scorer ou l'I/O est faux.

10.3 isolated_caches — sacs à dos indépendants

Chaque endpoint ne voit qu'un seul cache (endpoint e relié au cache $e \bmod C$). Il n'y a presque pas de recouvrement : le problème se découpe en C knapsacks séparés. Le greedy + DP est dans son cas d'école. Score : 597 826 (89,7 %). L'écart restant vient surtout de la capacité trop petite pour tout cacher, pas d'une mauvaise coordination entre caches.

10.4 trending_mini — copie de trending_today

Tous les endpoints voient tous les caches, avec la même latence 100 ms. Comme sur l'instance officielle trending, dupliquer une vidéo est inutile : une copie suffit pour tout le monde. Le gain marginal tombe à zéro dès la première copie. Score : 549 166, soit 98,1 % du plafond — l'un des meilleurs rapports mesurés.

10.5 small_mixed — style me_at_the_zoo

Petite instance hétérogène : latences datacenter variées, 1 à 5 caches par endpoint, tailles de vidéos entre 1 et 50 Mo, requêtes Zipf. C'est le terrain du greedy + knapsack, éventuellement hill-climb. Score : 435 295 (80,6

%). Plus difficile que trending car les caches se marchent dessus avec des latences différentes.

10.6 skewed_popular — popularité très concentrée

Zipf agressif (exposant 2,1) : quelques vidéos reçoivent presque toutes les requêtes, beaucoup d'autres sont ignorées. Le tri par densité place d'abord ces « hits » s'ils tiennent dans X. C'est exactement l'hypothèse du greedy. Score : 641 298, 98,8 % du plafond — meilleur cas généré avec trending_mini.

10.7 medium — encore le chemin précis

400 vidéos, 35 endpoints, 4 000 requêtes, 20 caches de 800 Mo. Assez grand pour être représentatif, encore sous le seuil d'expansion. On reste donc sur le greedy global et 2 tours de reopt, sans hill-climb ($V > 200$). Score : 510 403 (85,7 %). Profil intermédiaire, proche de videos en plus petit.

10.8 large_fastfill — déclenchement de _solve_large

800 vidéos, 60 endpoints, 90 000 requêtes, 45 caches, et surtout chaque endpoint est relié à tous les caches. L'expansion dépasse 8 millions, donc le fast-fill s'active. Malgré les approximations (70 endpoints max, pas de reopt), le score atteint 621 021, soit 97,9 % du plafond. La popularité reste assez concentrée (Zipf 1,4) et le graphe, bien que large, n'est pas aussi pathologique que kittens (1 000 endpoints, 500 caches, catalogue plus dur à couvrir).

10.9 Ce que ces instances apportent

Ensemble, elles forment une grille de lecture :

- no_cache et tiny_sanity valident le scorer et l'I/O.
- isolated_caches isole la qualité du knapsack, sans interaction.
- trending_mini et skewed_popular montrent le pic d'efficacité du gain marginal + densité.
- small_mixed et medium reproduisent le cas « caches qui se chevauchent ».
- large_fastfill prouve que le chemin rapide se déclenche et reste correct quand la structure est favorable.
- L'instance officielle kittens reste le vrai stress-test : même famille que large_fastfill, mais beaucoup plus dense et plus pauvre en capacité.

11. Conclusion

Le solveur repose sur une idée simple et adaptée au score officiel : ne jamais compter qu'un cache plus lent apporte quelque chose si un cache plus rapide a déjà la vidéo. Autour de cette idée, trois outils se complètent.

Le greedy à tas paresseux construit une première solution en plaçant les couples (cache, vidéo) de meilleure densité marginale. Le knapsack 0/1 réoptimise chaque cache une fois les autres figés. Le hill-climb rattrape encore quelques points sur les toutes petites instances. Quand le graphe devient trop grand, le fast-fill garantit une solution dans le temps du juge.

Les expériences le confirment. L'algorithme est quasi optimal sur les réseaux à latences cache égales et sur les catalogues à popularité très concentrée. Il reste solide sur les petites instances grâce à la DP et au hill-climb. Il perd surtout sur les instances énormes et denses, où l'on échange volontairement de la qualité contre du temps. Les huit instances générées permettent de retrouver ces régimes sans relancer kittens à chaque modification.

Annexe — commandes utiles

```
python main.py me_at_the_zoo.in me_at_the_zoo.out
python score.py me_at_the_zoo.in me_at_the_zoo.out
python main.py --all
python generate_instances.py --run
python main.py                # juge : stdin -> stdout
```